Applied Microeconometrics
Homework Assignment 1

Gerasimova Liudmila, Esina Victoria, Kliushinichenko Liya

April 3, 2026

This assignment replicates the analysis from Lee (2008), "Randomized experiments from non-random selection in U.S. House elections," estimating the incumbency advantage using a regression discontinuity design (RDD).

Task 1. Treatment Indicator

The treatment indicator `victory` equals 1 if the Democratic party won the election at time t, and 0 otherwise:

$$\texttt{victory}_i = \mathbf{1}\left[\texttt{difdemshare}_i \geq 0\right]$$

```
df['victory'] = (df['difdemshare'] >= 0).astype(int)
```

Result: Victory = 1: 6700 observations; Victory = 0: 5095 observations.

Assignment mechanism. This is a *sharp* regression discontinuity design. The assignment variable (running variable) is `difdemshare` - the Democratic vote share margin of victory in election t. Treatment (incumbency status) is assigned deterministically: the Democratic party becomes the incumbent if and only if the margin crosses the threshold of zero. The key identifying assumption is that, conditional on agents' choices and characteristics, the density of the vote share margin is continuous at the threshold. This implies that near the threshold, whether the Democrats win or lose is effectively determined by random chance (e.g., weather, turnout shocks), making treatment assignment locally as good as randomized.

Task 2. Local Averages Plot

The assignment variable is divided into bins of width 0.01 on both sides of the threshold, from -0.25 to 0.25. For each bin, the average value of `demsharenext` (Democratic vote share in election $t+1$) is computed and plotted.

```
df_plot = df[df['demsharenext'].notna()].copy()

bins_edges = np.arange(-0.255, 0.265, 0.01)
df_plot['bin'] = pd.cut(df_plot['difdemshare'], bins=bins_edges)
df_plot['bin_mid'] = df_plot['bin'].apply(
    lambda x: round(x.mid, 3) if pd.notna(x) else np.nan
)

local_avg = (
    df_plot[(df_plot['bin_mid'] >= -0.25) & (df_plot['bin_mid'] <= 0.25)]
    .groupby('bin_mid', observed=True)['demsharenext']
    .mean()
    .reset_index()
)
```

```
fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(local_avg['bin_mid'], local_avg['demshareneext'],
           s=30, color='steelblue', edgecolors='navy', linewidth=0.5, zorder=5)
ax.axvline(x=0, color='red', linestyle='--', alpha=0.7, label='Threshold (mov =
    0)')
ax.set_xlabel('Democratic Vote Share Margin of Victory, Election t', fontsize
    =12)
ax.set_ylabel('Democrat Vote Share, Election t+1', fontsize=12)
ax.set_title('Task 2: Local Averages of the Outcome Variable', fontsize=14)
ax.set_xlim(-0.27, 0.27)
ax.legend(fontsize=11)
plt.tight_layout()
plt.savefig('task2_local_averages.png', dpi=150)
plt.show()
```

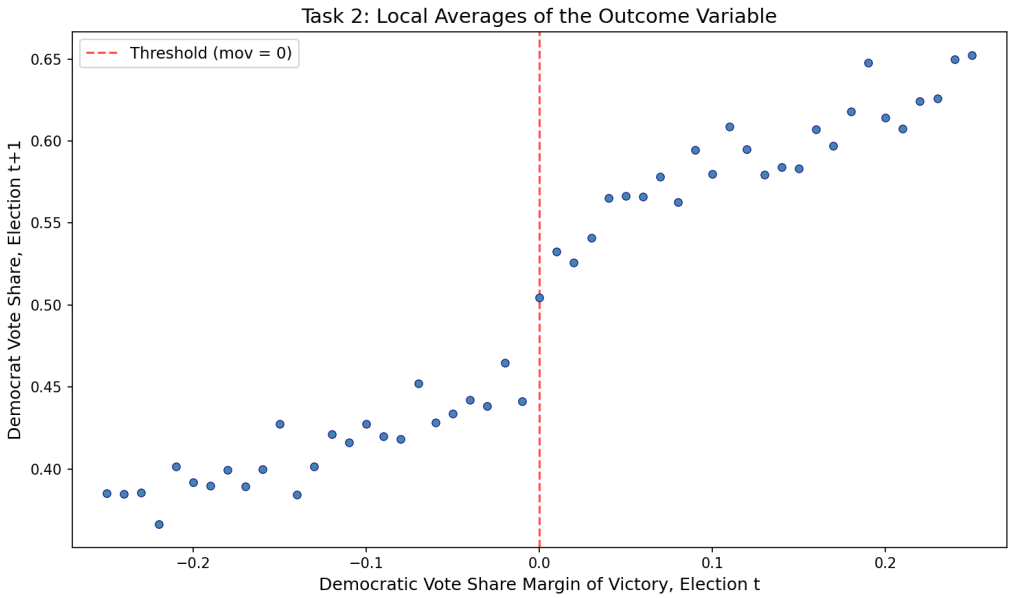


Figure 1: Local averages of the Democratic vote share in election $t+1$, by margin of victory in election t. Each point represents the average within a 0.01-wide bin.

Observation. There is a clear discontinuous jump at the threshold of zero. Democrats who barely won election t receive approximately 53% of the vote in election $t+1$, while Democrats who barely lost receive approximately 44%. This visual gap of about 9 percentage points provides preliminary evidence of a causal incumbency advantage. Away from the threshold, the relationship between the margin of victory and the future vote share is smooth and positive on both sides, reflecting selection: stronger candidates tend to both win by larger margins and perform better in subsequent elections.

Task 3. Polynomial Approximation

A fourth-order polynomial in the assignment variable is fitted separately on each side of the threshold, using the full sample (not just local averages):

```
df_work = df[df['demshareneext'].notna()].copy()

left  = df_work[df_work['difdemshare'] < 0]
right = df_work[df_work['difdemshare'] >= 0]
```

```
coeffs_left  = np.polyfit(left['difdemshare'],  left['demsharenext'],  4)
coeffs_right = np.polyfit(right['difdemshare'], right['demsharenext'], 4)


x_left  = np.linspace(-0.25, 0, 300)
x_right = np.linspace(0, 0.25, 300)
y_left_fit  = np.polyval(coeffs_left, x_left)
y_right_fit = np.polyval(coeffs_right, x_right)

fitted_left_at_0  = np.polyval(coeffs_left, 0)
fitted_right_at_0 = np.polyval(coeffs_right, 0)
gap = fitted_right_at_0 - fitted_left_at_0

fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(local_avg['bin_mid'], local_avg['demsharenext'],
           s=30, color='steelblue', edgecolors='navy', linewidth=0.5,
           zorder=5, label='Local averages')
ax.plot(x_left,  y_left_fit,  color='red', linewidth=2.5,
        label='4th-order polynomial fit')
ax.plot(x_right, y_right_fit, color='red', linewidth=2.5)
ax.axvline(x=0, color='gray', linestyle='--', alpha=0.5)
ax.set_xlabel('Democratic Vote Share Margin of Victory, Election t', fontsize
    =12)
ax.set_ylabel('Democrat Vote Share, Election t+1', fontsize=12)
ax.set_title('Task 3: Local Averages and Polynomial Fits', fontsize=14)
ax.set_xlim(-0.27, 0.27)
ax.set_ylim(0.30, 0.70)
ax.legend(fontsize=11)
plt.tight_layout()
```

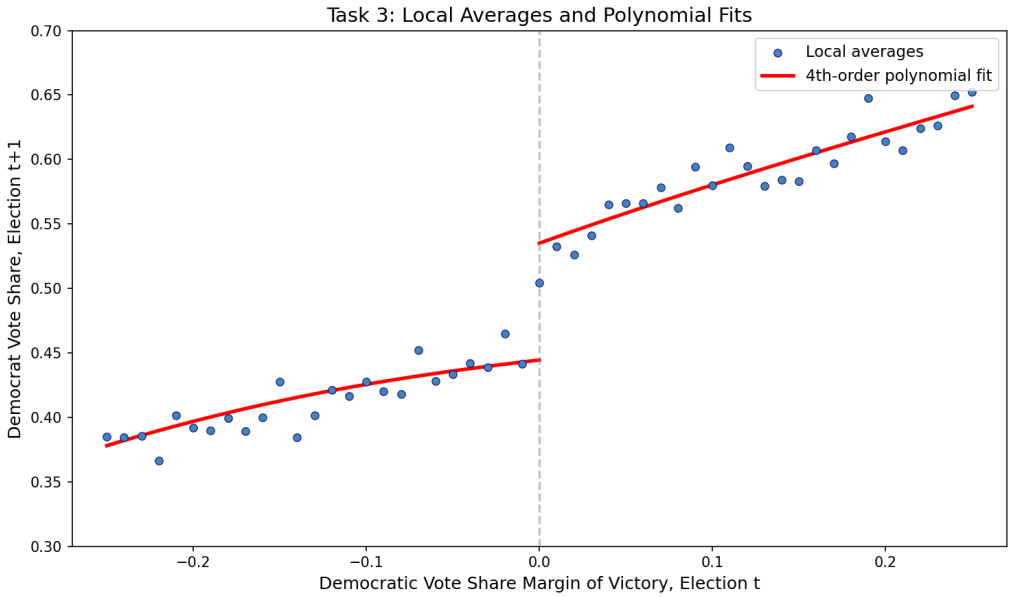


Figure 2: Local averages and fourth-order polynomial fits of the Democratic vote share in election $t+1$, by margin of victory in election t.

Discontinuity gap:

- Fitted value at 0 from the left: $\hat{y}^{-}(0) = 0.4445$
- Fitted value at 0 from the right: $\hat{y}^{+}(0) = 0.5351$
- Discontinuity gap: $\hat{y}^{+}(0) - \hat{y}^{-}(0) = 0.0906$

The polynomial fits track the local averages well. The estimated discontinuity gap of approximately 9.1 percentage points confirms the visual evidence from Task 2.

Task 4. RDD Regression Estimate

The incumbency effect is estimated by regressing the outcome on the victory indicator, a fourth-order polynomial in the margin of victory, and their interactions:

$$\texttt{demsharenext}_i = \beta_0 + \beta_1 \cdot \texttt{victory}_i + f(\texttt{mov}_i) + \texttt{victory}_i \cdot f(\texttt{mov}_i) + \varepsilon_i$$

where $f(\texttt{mov}) = \gamma_1 \texttt{mov} + \gamma_2 \texttt{mov}^2 + \gamma_3 \texttt{mov}^3 + \gamma_4 \texttt{mov}^4$. This specification is equivalent to fitting separate fourth-order polynomials on each side of the threshold. Standard errors are clustered at the state-district-decade level.

```
df_reg = df_work.copy()
mov = df_reg['difdemshare']
vic = df_reg['victory']

df_reg['mov2']     = mov ** 2
df_reg['mov3']     = mov ** 3
df_reg['mov4']     = mov ** 4
df_reg['vic_mov']  = vic * mov
df_reg['vic_mov2'] = vic * mov ** 2
df_reg['vic_mov3'] = vic * mov ** 3
df_reg['vic_mov4'] = vic * mov ** 4

X_vars = ['victory', 'difdemshare', 'mov2', 'mov3', 'mov4',
          'vic_mov', 'vic_mov2', 'vic_mov3', 'vic_mov4']

X4 = sm.add_constant(df_reg[X_vars])
y4 = df_reg['demsharenext']

model4 = sm.OLS(y4, X4).fit(
    cov_type='cluster',
    cov_kwds={'groups': df_reg['statedisdec']}
)

for var in ['const'] + X_vars:
    print(f"{var:<20} {model4.params[var]:>10.4f} {model4.bse[var]:>10.4f} "
          f"{model4.tvalues[var]:>10.3f} {model4.pvalues[var]:>10.4f}")
```

Table 1: RDD estimate of the incumbency effect (Task 4)

Variable	Coef.	Std. Err.	z	$P > \lvert z \rvert$
Constant	0.4445	0.0076	58.789	0.0000
Victory	**0.0906**	**0.0102**	**8.905**	**0.0000**
mov	0.1561	0.1366	1.143	0.2531
mov^2	−0.2175	0.7052	−0.308	0.7578
mov^3	1.2444	1.2876	0.966	0.3338
mov^4	1.4518	0.7238	2.006	0.0449
victory × mov	0.3256	0.1773	1.837	0.0663
victory × mov^2	−0.1368	0.8551	−0.160	0.8729
victory × mov^3	−0.6542	1.5256	−0.429	0.6680
victory × mov^4	−1.8198	0.8248	−2.206	0.0274
N		9177		
R^2		0.6700		

Interpretation. The coefficient on `victory` is $\hat{\beta}_1 = 0.091$ (s.e. $= 0.010$, $p < 0.001$). This represents the causal effect of incumbency: winning election t increases the Democratic party's vote share in election $t+1$ by approximately **9.1 percentage points**. The coefficient is the estimated discontinuity gap at the threshold, identified under the assumption that treatment is locally randomized among closely contested elections. The interaction terms allow the polynomial to take different shapes on each side of the threshold, ensuring that the discontinuity gap is not driven by functional form assumptions.

Task 5. Sensitivity to Baseline Covariates

The following pre-determined covariates are added to the specification from Task 4: `demshareprev`, `demwinprev`, `demofficeexp`, `demelectexp`, `othofficeexp`, `othelectexp`.

```
covariates = ['demshareprev', 'demwinprev', 'demofficeexp',
              'demelectexp', 'othofficeexp', 'othelectexp']

df_reg5 = df_reg.dropna(subset=covariates).copy()
y5 = df_reg5['demsharenext']

X5 = sm.add_constant(df_reg5[X_vars + covariates])
model5 = sm.OLS(y5, X5).fit(
    cov_type='cluster',
    cov_kwds={'groups': df_reg5['statedisdec']}
)

for var in ['const'] + X_vars + covariates:
    print(f"{var:<20} {model5.params[var]:>10.4f} {model5.bse[var]:>10.4f} "
          f"{model5.tvalues[var]:>10.3f} {model5.pvalues[var]:>10.4f}")
```

Table 2: RDD estimate with baseline covariates (Task 5)

Variable	Coef.	Std. Err.	z	$P > \|z\|$
Constant	0.3132	0.0111	28.293	0.0000
Victory	**0.0766**	**0.0109**	**7.025**	**0.0000**
mov	0.4247	0.1375	3.088	0.0020
mov^2	1.2294	0.7003	1.756	0.0792
mov^3	3.2348	1.2588	2.570	0.0102
mov^4	2.3043	0.7011	3.287	0.0010
victory × mov	−0.0357	0.1872	−0.191	0.8487
victory × mov^2	−1.6073	0.8982	−1.789	0.0735
victory × mov^3	−2.6305	1.5102	−1.742	0.0815
victory × mov^4	−2.6764	0.8367	−3.199	0.0014
demshareprev	0.2980	0.0171	17.420	0.0000
demwinprev	−0.0066	0.0070	−0.937	0.3489
demofficeexp	−0.0002	0.0027	−0.060	0.9519
demelectexp	−0.0029	0.0026	−1.106	0.2687
othofficeexp	−0.0002	0.0037	−0.064	0.9494
othelectexp	0.0033	0.0035	0.944	0.3451
N	6559			
R^2	0.7117			

Comparison. On the same restricted sample ($N = 6{,}559$), the incumbency effect without covariates is 0.0755 (s.e. $= 0.0114$), and with covariates it is 0.0766 (s.e. $= 0.0109$). The estimate

is virtually unchanged.

Interpretation. The stability of the incumbency effect across specifications is exactly what we would expect if the RDD is valid. Under Proposition 3 of Lee (2008), near the threshold treatment status is locally randomized, so all pre-determined characteristics are independent of treatment. Therefore, including baseline covariates should not substantially alter the point estimate - they can only reduce sampling variability. Indeed, the only covariate with a statistically significant coefficient is `demshareprev` (0.298, $p < 0.001$), which helps predict the outcome but does not confound the treatment effect. The R^2 increases from 0.67 to 0.71, while the standard error on `victory` slightly decreases, confirming that covariates improve precision without changing the estimate. This robustness serves as an additional specification test supporting the validity of the RDD.